Automotive Innovation (2026) 9:158–174
https://doi.org/10.1007/s42154-025-00369-5

A 4WIS Vehicle Trajectory Planning Approach Incorporating Multiple Motion Mode Sequence Decision-Making

Zeyu Yang[1,2] · Yuan Chang[1] · Yingjie Jin[1] · Mingyang Chen[3] · Manjiang Hu[1,2] · Yougang Bian[1,2] · Xiaoyan Peng[1]

Received: 17 February 2025 / Revised: 17 February 2025 / Accepted: 25 February 2025 / Published online: 13 January 2026
© China Society of Automotive Engineers (China SAE) 2026

Abstract
Four-wheel independent steering (4WIS) vehicles possess remarkable flexibility due to their multiple motion modes. However, efficiently allocating these motion modes to reach the destination under various operating conditions poses a significant challenge in trajectory planning. To address this issue, this study proposes a 4WIS vehicle trajectory planning method incorporating multiple motion mode sequence decision-making. The proposed method aims to simultaneously achieve optimal motion mode sequence decision-making and trajectory planning in diverse environments. Firstly, a multi-mode collaborative planning algorithm based on graph search is developed. This algorithm generates reasonable global paths by employing strategies such as multi-mode node expansion, optimal node selection, and goal node connection. Subsequently, the global path is segmented, and then the each path segment is optimized using the quadratic programming (QP) method. Finally, the trapezoidal velocity planning method is applied to the optimized path to generate the final trajectory. Simulations demonstrate that the proposed algorithm automatically determines an optimal sequence of motion modes while generating safe and smooth trajectories. Notably, in complex environments, the proposed method significantly enhances the vehicle's trafficability.

Keywords 4WIS vehicle · Trajectory planning · Multiple motion modes · Graph search algorithm

1 Introduction

With the growing number of vehicles and the expansion of urban infrastructure, driving spaces have become increasingly constrained, particularly in unstructured environments like parking lots and squares. In such congested settings, the four-wheel independent steering (4WIS) system presents a promising solution facilitating vehicle mobility. In contrast to traditional front-wheel steering vehicles, 4WIS vehicles possess the ability to independently control all four wheels, enabling unique motion modes such as diagonal-move and in-place rotation [1]. These additional motion modes provide 4WIS vehicles with enhanced maneuverability [2], which is vital for navigating in narrow environments and effectively alleviating traffic congestion.

Intelligentization is a significant trend in the advancement of vehicle technology, and research on intelligent technology for 4WIS vehicles has also received considerable attention [3, 4]. Lai et al. [5] developed kinematic and dynamic models for typical motion modes in 4WIS vehicles, and formulated motion mode switching control strategies. Chen et al. [6] examined the dynamic switching logic among three motion modes: front-wheel steering, rear-wheel steering, and four-wheel Ackermann steering, and developed a motion mode switching control algorithm. Significant progress has been made in vehicle control for 4WIS vehicles. However, there has been limited exploration and a lack of mature solutions specifically related to trajectory planning [7]. Therefore, it is crucial to conduct research on planning methods for 4WIS vehicles to achieve their intelligent capabilities. The complexity of the multi-motion mode configuration in 4WIS vehicles presents substantial challenges for trajectory planning. To address this, this paper investigates trajectory planning for 4WIS vehicles, with a particular emphasis on leveraging their high maneuverability.

✉ Manjiang Hu
manjiang_h@hnu.edu.cn

[1] State Key Laboratory of Advanced Design and Manufacturing Technology for Vehicle, Hunan University, Changsha 410082, China

[2] Wuxi Intelligent Control Research Institute of Hunan University, Wuxi 214072, China

[3] University College London, London WC1E 6BT, UK

At present, there are four commonly used trajectory planning methods [8–10]: the sampling-based method, the curve interpolation method, the numerical optimization method, and the search-based method. The sampling-based method can quickly identify the goal node by sampling in the free configuration space and evaluating the feasibility of each sampled node. It mainly includes rapidly-exploring random tree (RRT) [11], probabilistic roadmaps (PRM) [12], and their corresponding variants [13, 14]. Wang et al. [15] proposed an efficient algorithm based on bidirectional RRT. This algorithm utilizes the backward tree as a heuristic to guide the growth of the forward tree, thereby effectively circumventing the two-point boundary value problem encountered during the connection of the two trees. The aforementioned sampling-based method exhibits notable advantages in solving trajectory planning problems in large-scale spaces. The curve interpolation method effectively generates collision-free paths by incorporating curve segments between path nodes. It employs a variety of line combinations, including Dubins [16], Reeds-Shepp (R-S)[17], polynomial [18], B-spline [19], and Bezier [20] curves, to ensure smoothness and continuity. For instance, Wang et al. [21] proposed a lane changing trajectory planning method based on the quintic polynomial, which can obtain safe and comfortable lane changing trajectories. In comparison to other methods, this curve interpolation method satisfactorily fulfills the requirements of path smoothness, continuity, and controllability. However, it is more suitable for structural scenes such as highways. The numerical optimization method comprehensively considers the objectives function and constraints conditions of the trajectory planning problem. It formulates the problem as an optimal control problem (OCP), and employs transformation and solving techniques to derive the optimal solution. For example, Zhou et al. [22] approximated the original nonlinear programming (NLP) problem as a quadratic programming (QP) subproblem[23], which improved the computational efficiency. Zhang et al. [24] accurately reformulated non-differentiable collision avoidance constraints into smooth, differentiable constraints using the strong duality of convex optimization, enabling real-time optimization-based trajectory planning in tight environments. While these numerical optimization methods have the capability to derive theoretical optimal solutions, they typically rely on a good initial value provided by another method and are often used in conjunction with search-based methods [25].

The search-based planning method typically traverse a state space to discover a path that connects the starting point and the end point. This state space is often represented as an occupancy grid, which describes the object locations in the environment. The primary objectives of this method are to find the shortest path and achieve optimal efficiency. Notable methods within this category include Dijkstra [26], A* [27], Hybrid-A* [28], and their corresponding variants [29, 30]. The A* algorithm can search for feasible paths between two nodes while avoiding obstacles by setting the motion primitive as a straight line, which is suitable for omnidirectional robots. Based on the A* planning method, ARA* [31] and D* [32] better adapt to dynamic environments. To solve the path planning problem of vehicles with kinematic constraints, the Hybrid-A* planner [28] combines the two-dimensional A* [27] and the kinematics of the front-wheel steering vehicle. Zhang et al. [33] divided a complex automated valet parking path planning into several simple path plans, which makes the Hybrid-A* algorithm more applicable in the complex parking environment. This type of search-based method can provide a good initial solution for the numerical optimization method and has strong adaptability to different scenarios and kinematic constraints.

The above trajectory planning methods solely focus on a single motion mode, thereby limiting their ability to fully exploit the potential of multiple motion modes in 4WIS vehicles. Currently, there is limited research on exploring the characteristics of multiple motion modes in 4WIS vehicles. For instance, Zhang et al. [34] proposed an enhanced hybrid space KIECEP (HS-KIECEP) method for 4WIS robots equipped with multiple motion modes. This method performs a sampling-based planning to generate paths. During each iteration, a motion mode is randomly selected to extend the state tree. Moreover, this method takes into account multiple motion modes and kinematic constraints, enabling the generation of feasible paths. Furthermore, Xu et al. [35] utilized b-spline theory to plan the motion path of the 4WIS rescue vehicle and employed a genetic algorithm for path smoothing optimization. While this planning method incorporates the front-wheel steering and rear-wheel steering motion modes, it overlooks crucial diagonal-move and in-place rotation modes. Although these studies have explored the characteristics of multiple motion modes in 4WIS vehicles, they still have certain limitations. They either employ random motion mode selection or neglect crucial specialized motion modes. As a result, they may generate unreasonable motion mode sequences, thereby impeding the full utilization of the 4WIS system's high flexibility.

Search-based planning methods offer a reliable framework for trajectory planning, demonstrating extensive applicability and flexibility. By designing motion primitives and cost functions, it can effectively handle diverse vehicle constraints and planning objectives. For example, the classic Hybrid-A* algorithm solves the planning problem of front-wheel steering vehicles with kinematic constraints. Similarly, by integrating the kinematic constraints of 4WIS vehicles, it is feasible to use search-based algorithms for path planning in 4WIS vehicles.

Based on the preceding discussion, this paper presents a 4WIS vehicle trajectory planning method incorporating

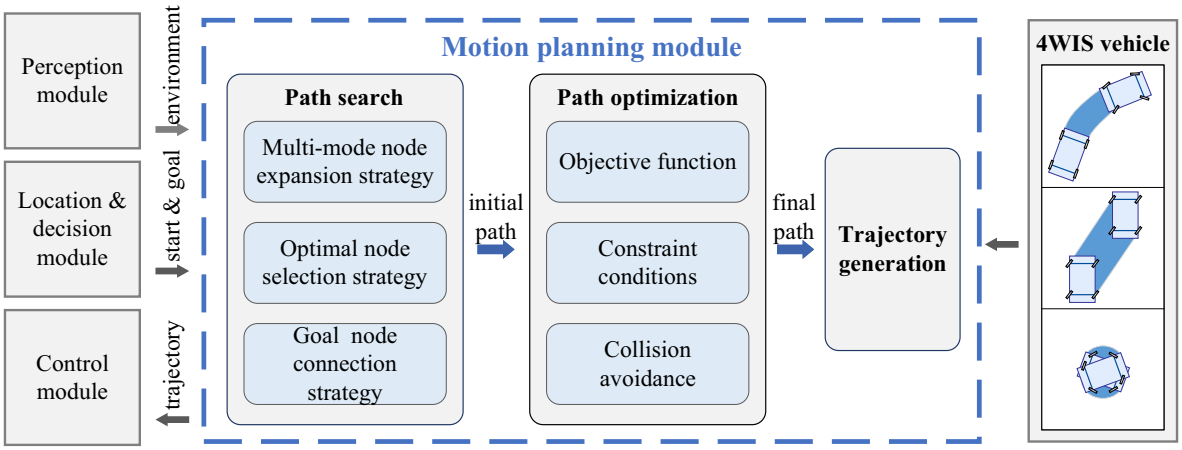


Fig. 1 System framework of the proposed trajectory planning method

multiple motion mode sequence decision-making, which aims to make full use of multiple motion modes for trajectory planning. This method combines search-based and numerical optimization approaches, which is implemented in three stages, i.e, path search, path optimization, and trajectory generation. In the path search part, the path is searched on the discrete grid map using a multi-mode collaborative planning algorithm. Guided by the optimal node selection strategy, the use of multi-mode node expansion strategy and goal node connection strategy can ensure an optimal motion modes sequence in the search process and accelerate the search speed. Once the initial path is found, QP based numerical optimization methods are employed in the path optimization step. By designing an objective function and imposing constraint conditions, the path can be optimized. Additionally, collision check is performed to ensure the safety of the optimized path. Finally, a speed profile is attached to the path to obtain the final trajectory.

The contribution of this study mainly includes:

(1) A graph search method considering multiple motion modes is proposed. By carefully designing the node cost function and node selection strategy, the method successfully integrates the multi-motion modes of 4WIS vehicles into the graph search algorithm. Compared to path planning methods designed for a single motion mode, the proposed approach not only generates optimal paths but also produces a sequence of optimal motion modes, fully exploiting the flexibility of 4WIS vehicles.
(2) A segment-wise path optimization strategy that accounts for multiple motion modes is developed. By relaxing the constraint conditions related to in-place rotation endpoints, the smoothness and comfort of the optimized path can be significantly improved. Compared to the work in Ref. [35], the proposed method achieves optimality, attributed to the designed node selection strategy and the numerical optimization for path.

The remaining sections of this study are organized as follows. Section 2 introduces the overall framework of the system. Section 3 describes the preliminary knowledge, including the kinematics model of the 4WIS vehicle, the establishment of search space and the implementation of collision check. Sections 4 and 5 provide a detailed description of the primary design processes of the proposed trajectory planning algorithms. Section 6 presents simulation results and analysis. Section 7 draws the conclusion.

2 System Framework

The system framework of the proposed trajectory planning method is depicted in Fig. 1. The perception module gathers information about the surrounding environment and provides it to the planning module, which is then utilized to generate the planning map. The location module supplies the information of the starting node, while the decision module provides the information of the goal node. Based on the aforementioned modules and 4WIS vehicle kinematics, the trajectory planner can generate a feasible trajectory from the start node to the goal node, which is then passed to the control module. The control module is responsible for executing trajectory tracking and controlling the vehicle to reach the destination.

The proposed trajectory planner consists of three parts: path search, path optimization, and trajectory generation.

(1) In the first stage, the proposed multi-mode collaborative planning algorithm is used to generate the initial path and motion mode sequence. This algorithm employs

a multi-mode node expansion strategy that encompasses a variety of motion primitives, thereby greatly enhancing its applicability to complex environments. At the same time, the optimal node selection strategy improves the rationality of the motion mode sequence by using the multi-objective node cost and node selection method to guide the search process. Then, the goal node connection strategy is employed to generate multiple fitting paths that connect the selected node and the goal node. The optimal fitting path is then selected, resulting in an improved connection success rate and path smoothness.

(2) In the second stage, the QP algorithm is used to optimize the initial path to improve the smoothness of the path. At this stage, the setting of constraints takes into account the mode switching conditions at the endpoint, which can greatly improve the smoothness of the path at the in-place rotation endpoint. Moreover, a cyclic optimization method is applied to ensure the security of the optimized path.

(3) In the third stage, the trapezoidal speed planning method is utilized to plan the speed of the optimized path to generate the final trajectory.

3 Preliminary Knowledge

3.1 Vehicle Model

The 4WIS vehicle incorporates multiple motion modes. Due to the intricacy of these motion modes, the bi-steerable model with two virtual wheels is employed to depict the vehicle's kinematics in each mode. The vehicle state $\boldsymbol{s}$ can be expressed as follows:

$$\boldsymbol{s} = [x, y, \theta, \delta, \upsilon]^{\top} \tag{1}$$

where (x, y) denote the coordinates of vehicle center and θ is the orientation of vehicle body in the referenced frame; δ represents the wheel rotation angle; υ expresses the vehicle speed.

3.1.1 Four-Wheel Ackermann Steering Mode

As shown in Fig. 2(a), the four-wheel Ackermann steering mode can be simplified to a bicycle model with two virtual wheels. In this simplified model, by assuming that the turning angles of the front and rear wheels are equal but opposite in direction, the minimum turning radius can be achieved, thereby fully utilizing the vehicle's flexibility. The vehicle speed υ is determined by the wheel speed u, and the relationship is as follows:

$$\upsilon = u \cos \delta \tag{2}$$

The vehicle in four-wheel Ackermann steering mode follows the kinematics equation [4]:

$$\dot{\boldsymbol{s}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\delta} \\ \dot{\upsilon} \end{bmatrix} = \begin{bmatrix} \upsilon \cos\theta \\ \upsilon \sin\theta \\ 2\upsilon \tan\delta / L \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ -\upsilon \tan\delta & \cos\delta \end{bmatrix} \begin{bmatrix} \omega \\ a \end{bmatrix} \tag{3}$$

where L is the wheelbase; ω is the angular velocity; $a = \dot{u}$ denotes the acceleration of wheel.

3.1.2 Diagonal-Move Mode

Fig. 2(b) shows the vehicle model in diagonal-move mode. The four wheels of the vehicle are turned to the same angle, enabling the vehicle to move diagonally. In this motion mode, the relationship between υ and u is as follows:

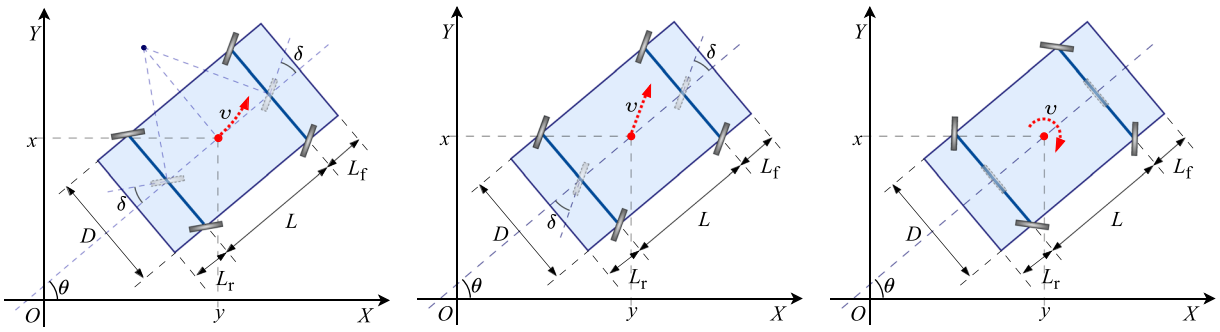


Fig. 2 Schematics of vehicle kinematic model. **a** Four-wheel Ackermann steering mode; **b** Diagonal-move mode; **c** In-place rotation mode

$$\upsilon = u \tag{4}$$

The vehicle in diagonal-move mode follows the kinematics equation [36]:

$$\dot{\boldsymbol{s}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\delta} \\ \dot{\upsilon} \end{bmatrix} = \begin{bmatrix} \upsilon \cos(\theta + \delta) \\ \upsilon \sin(\theta + \delta) \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \omega \\ a \end{bmatrix} \tag{5}$$

3.1.3 In-Place Rotation Mode

Fig. 2(c) shows the vehicle model in in-place rotation mode. In this mode, each wheel will rotate to a specified angle, enabling the vehicle to rotate around its center. The absolute value of the wheel speed u is fixed, and its direction determines whether the vehicle rotates clockwise or counterclockwise. The vehicle in in-place rotation mode follows the kinematics equation [34]:

$$\dot{\boldsymbol{s}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\delta} \\ \dot{\upsilon} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 2u/\sqrt{D^2 + L^2} \\ 0 \\ 0 \end{bmatrix} \tag{6}$$

where D is the vehicle width; vehicle speed $\upsilon = 0$.

The absolute values of the four wheel rotation angles δ are determined by

$$|\delta| = \arctan \frac{L}{D} \tag{7}$$

Remark 1 To simplify the description of in-place rotation without introducing wheel speed u as a state variable, this study assumes a fixed wheel speed during in-place rotation. This assumption does not affect the path planning process because the planned path is fully determined as long as the heading angles at the start and end of the rotation are specified. The variation in wheel speed during the rotation can be omitted during path planning. For the in-place rotation mode, the precise wheel speed control commands can be calculated during the trajectory tracking phase rather than at the path planning or trajectory planning stages.

Remark 2 In practice, beyond the three motion modes illustrated in Fig. 2, 4WIS vehicles are capable of a broader range of motion modes. However, these three modes are sufficient to address the trajectory planning requirements in the vast majority of scenarios. This study assumes that the vehicle operates in only these three representative motion modes, which are sufficient to demonstrate the flexibility advantage of 4WIS vehicles over front-wheel steering counterparts. The resulting planned path or trajectory may trade off some optimality for reduced computational burden.

3.2 Grid Map and Collision Check

Before planning, the scene is discretized into a grid map, where each cell represents a specific area in the environment. In this grid map, we utilize occupied grid cells to represent obstacles. During the planning phase, the grid graph is utilized to perform the collision check and ensure the safety of each path node. The vehicle is defined as a rectangle with a total length $\left(L + L_{\mathrm{f}} + L_{\mathrm{r}}\right)$ and a width D. For each path node state $\mathbf{s}$, the vehicle rectangle is projected onto the corresponding position (x, y) and orientation θ on the grid map. The collision check is then conducted by examining whether the grid cells within the rectangular range are occupied by obstacles. In addition, to ensure sufficient safety, the vehicle rectangle is expanded by a safe distance around its boundaries. This expansion accounts for potential uncertainties or errors in the perception and control systems, providing an additional buffer to avoid collisions and ensure the safety of the planned path.

4 Multi-mode Collaborative Planning Algorithm

The multi-mode collaborative planning algorithm is presented in Algorithm 1 and Fig. 3. It encompasses several sub-processes detailed in Sections 4.1, 4.2, and 4.3. Similar to the traditional graph search algorithm, this algorithm searches the path in the discrete three-dimensional space corresponding to the vehicle's position and orientation (x,y,θ). Moreover, the algorithm constructs two sets of nodes: the open set $\mathcal{O}$ stores the nodes discovered during expansion, and the close set $\mathcal{C}$ stores the nodes that have already been selected and expanded. During each iteration of the search, the algorithm selects the current node $\boldsymbol{s}_{\mathrm{curr}}$ from $\mathcal{O}$ using the optimal node selection strategy. A series of motion primitives are then added through the multi-mode node expansion strategy, generating a set of child nodes. The collision check is performed on each child node using the occupied grid map $\mathcal{M}$ to identify safe child nodes $\boldsymbol{s}_{\mathrm{child}}$. For each $\boldsymbol{s}_{\mathrm{child}}$, the algorithm calculates its multi-objective node cost $f_{\mathrm{cost}}(\boldsymbol{s}_{\mathrm{child}})$ and designates the current node $\boldsymbol{s}_{\mathrm{curr}}$ as its parent node. When searching approaching the goal node $\boldsymbol{s}_{\mathrm{goal}}$, the goal node connection strategy is employed to search for the goal node. Finally, the algorithm conducts path backtracking from the recorded parent node to complete the entire path search, resulting in the determination of the path $\mathcal{P}$.

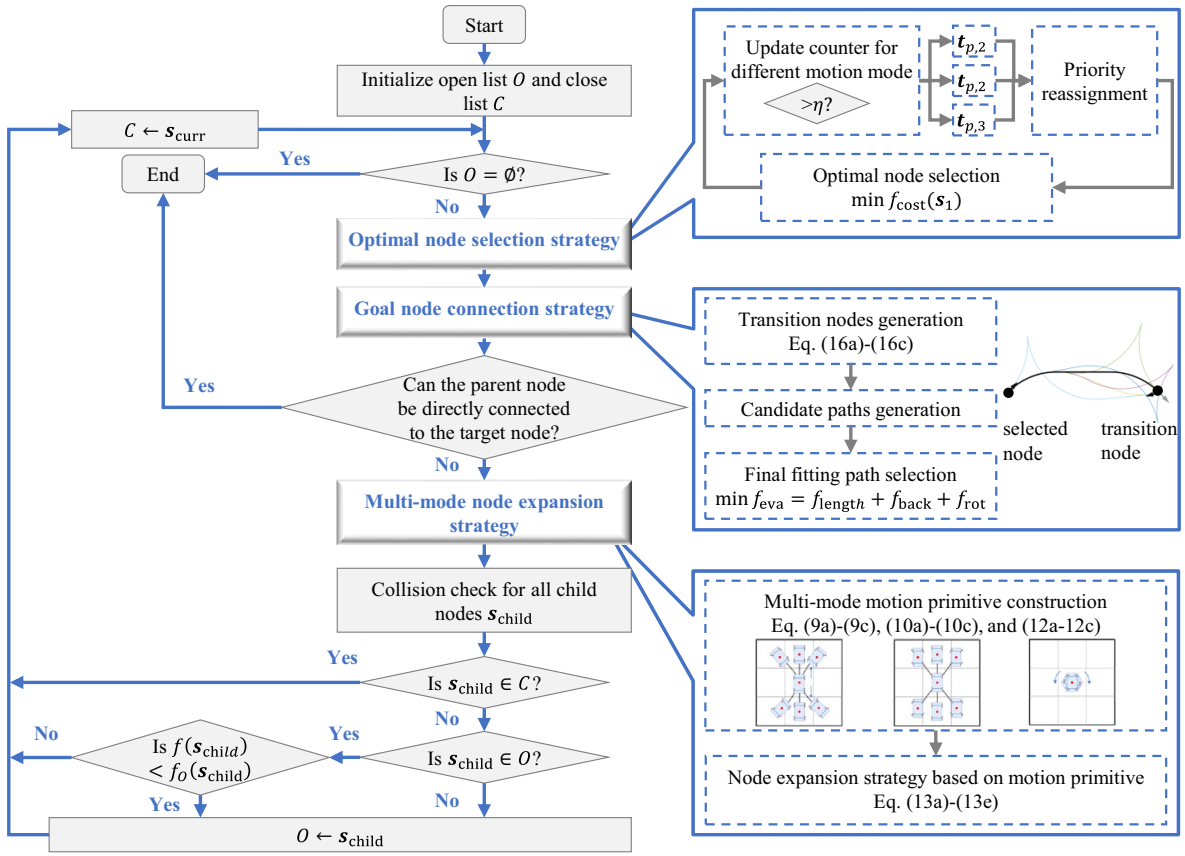


Fig. 3 System framework of the proposed multi-mode collaborative planning method

Algorithm 1 Proposed Graph Search Algorithm.

Input: $\mathcal{M}, s_{start}, s_{goal}$
Output: $\mathcal{P}$
Initialize $O = \emptyset$ and $C = \emptyset$
$O \leftarrow s_{start}$
while $O \neq \emptyset$ **do**
  $s_{curr} \leftarrow$ `OptimalNodeSelection`(O)
  if `GoalConnection`(s_{curr}) **then**
    $\mathcal{P} \leftarrow$ `PathBacktracking`(s_{goal})
    return $\mathcal{P}$
  for each $s_{child} \in$ `MultiModeExpansion`(s_{curr}) **do**
    $f(s_{child}) \leftarrow$ `MultiObjectCost`(s_{child})
    if $s_{child} \notin C$ and $s_{child} \notin O$ **then**
      $O \leftarrow s_{child}$
    else if $s_{child} \in O$ and $f(s_{child}) < f_O(s_{child})$ **then**
      $O \leftarrow s_{child}$
  $C \leftarrow s_{curr}$

4.1 Multi-mode Node Expansion Strategy

In the function `MultiModeExpansion()` of Algorithm 1, node expansion is primarily accomplished by utilizing a series of motion primitives generated offline. The generation of motion primitives for front-wheel steering vehicles is relatively straightforward and primarily dependent on the wheel rotation angle. However, it becomes more complex for 4WIS vehicles, which possess multiple motion modes that allow for a wide range of diverse motion primitives. Therefore, we have designed specific motion primitives for each motion mode of the 4WIS vehicle, as illustrated in Fig. 4. These motion primitives encompass various motion modes of the 4WIS vehicle, allowing it to adapt to different planning environments. Furthermore, when calculating the motion primitives, the vehicle speed is normalized to ± 1 to represent the driving direction. A value of $+1$ represents forward motion, while -1 represents backward motion.

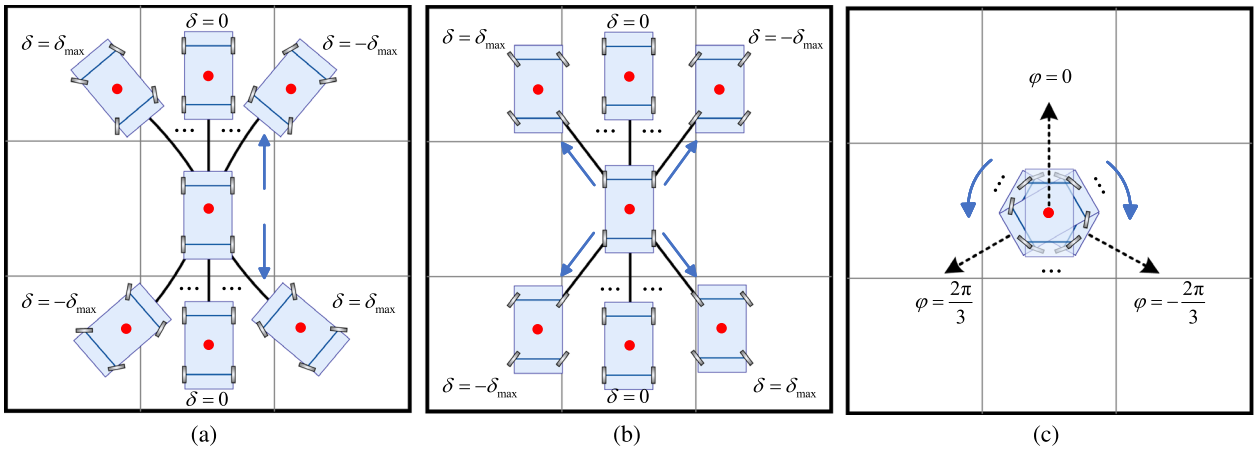


Fig. 4 Motion primitives of each motion mode: **a** Four-wheel Ackermann steering mode; **b** Diagonal-move mode; **c** In-place rotation mode

4.1.1 Motion Primitives of the Four-Wheel Ackermann Steering

The motion primitive of the four-wheel Ackermann steering mode is determined by parameters $\{l, \upsilon, \delta\}$, where l is the extension length, δ is the front wheel rotation angle, and several optional angle values are obtained by discretizing the rotation angle range, that is

$$\delta = -\delta_{\max} + i \cdot \frac{2\delta_{\max}}{n_1 - 1}, i \in \mathbf{I}_0^{n_1 - 1} \tag{8}$$

where $\delta_{\max}$ and $\delta_{\min}$ represent the maximum and minimum wheel rotation angles; n_1 represents the number of discrete wheel rotation angle. $\mathbf{I}_a^b$ denotes the set of positive integers from a to b. Specifically, in Eq. (8), $a = 0$ and $b = n_1 - 1$.

A series of motion primitives are generated by combining different parameters. In the context of path planning, speed variations are temporarily disregarded by assuming that the velocity remains constant over a sufficiently small time interval [0, l]. According to Eq. (3), these motion primitives can be obtained by the following integration formula for l:

$$\Delta\theta(l) = \int_0^l \upsilon \cdot \frac{2\tan\delta}{L} \mathrm{d}l \tag{9a}$$

$$\Delta x(l) = \int_0^l \upsilon \cos\left(\Delta\theta(l)\right) \mathrm{d}l \tag{9b}$$

$$\Delta y(l) = \int_0^l \upsilon \sin\left(\Delta\theta(l)\right) \mathrm{d}l \tag{9c}$$

4.1.2 Motion Primitives of the Diagonal-Move Mode

Similarly, the motion primitive of the diagonal-move mode is also determined by the parameters $\{l, \upsilon, \delta\}$, but with the number of discrete wheel rotation angle represented by n_2 instead of n_1. The front wheel rotation angle δ of the vehicle can be obtained from Eq. (8). Similar to the Ackermann steering mode, the velocity is assumed to remain constant over the time interval [0, l]. According to Eq. (5), the generation of motion elements can be obtained by the following formula:

$$\Delta\theta(l) = 0 \tag{10a}$$

$$\Delta x(l) = \int_0^l \upsilon \cos\left(\delta\right) \mathrm{d}l \tag{10b}$$

$$\Delta y(l) = \int_0^l \upsilon \sin\left(\delta\right) \mathrm{d}l \tag{10c}$$

4.1.3 Motion Primitives of the In-Place Rotation Mode

Motion primitives of the in-place rotation mode is determined by parameters $\{u, \varphi\}$, where wheel velocity u is normalized to ± 1 to indicate the direction of rotation, vehicle speed υ is 0; φ is the rotation angle, and its set of values is calculated as follows:

$$\varphi = i \cdot \frac{2\pi}{n_3}, i \in \mathbf{I}_1^{n_3} \tag{11}$$

where n_3 represents the number of angle values.

According to Eq. (6), the generation of motion primitives can be obtained by the following formula:

$$\Delta\theta(l) = u \cdot \varphi \tag{12a}$$

$$\Delta x(l) = 0 \tag{12b}$$

$$\Delta y(l) = 0 \tag{12c}$$

4.1.4 Node Expansion Strategy Based on Motion Primitives

With the translation and rotation transformations, motion primitives can be attached to any node $\boldsymbol{s}_0$. According to the parameters of the motion primitive, the child node $\boldsymbol{s}_1$ can be obtained as follows:

$$\theta_1 = \theta_0 + \Delta\theta(l) \tag{13a}$$

$$x_1 = x_0 + \Delta x(l)\cos\theta_0 - \Delta y(l)\sin\theta_0 \tag{13b}$$

$$y_1 = y_0 + \Delta x(l)\sin\theta_0 + \Delta y(l)\cos\theta_0 \tag{13c}$$

$$\delta_1 = \delta \tag{13d}$$

$$\upsilon_1 = \upsilon \tag{13e}$$

where δ and υ are determined by the parameters of the motion primitives.

When adding motion primitives to generate child nodes, it is crucial to perform the collision check to ensure safety. Since our collision check is directly performed at the child node position, it is essential to account for the safety of the vehicle during the transition between node states. When adding four wheel Ackermann steering motion primitives and diagonal-move mode motion primitives to the node, we set a small step size l to ensure the safety between the two nodes. However, when it comes to the motion primitives of the in-place rotation mode, some of them involve significant changes in the vehicle orientation θ. If these motion primitives are directly attached, we would not be able to guarantee safety during the vehicle rotation process. To overcome this issue, we adopt a progressive approach when attaching the in-place rotation motion primitives. We start by considering smaller changes in θ and gradually increase the change amount. During this attachment process, if a collision is detected, we halt the attachment of motion primitives of the in-place rotation mode with a larger change in θ.

4.2 Optimal Node Selection Strategy

4.2.1 Multi-objective Node Cost

The node cost function should not only ensure the optimal motion mode sequence, but also ensure the path quality in each motion mode. According to whether the vehicle motion mode is changed, the multi-objective node cost function is calculated as follows:

$$f_{\text{cost}}(\boldsymbol{s}_1) = \begin{cases} g_{\text{hold}}(\boldsymbol{s}_1, \boldsymbol{s}_0) + h(\boldsymbol{s}_1) & , \ p_1 = p_0 \\ g_{\text{switch1}}(\boldsymbol{s}_1, \boldsymbol{s}_0) + & \\ \quad g_{\text{switch2}}(\boldsymbol{s}_1, \boldsymbol{s}_0) + h(\boldsymbol{s}_1) & , \ p_1 \neq p_0 \end{cases} \tag{14}$$

where $\boldsymbol{s}_0$ and $\boldsymbol{s}_1$ denote the father node and child node; $g_{\text{hold}}(\boldsymbol{s}_1, \boldsymbol{s}_0)$ denote the cost function of holding motion modes; $g_{\text{switch1}}(\boldsymbol{s}_1, \boldsymbol{s}_0)$ and $g_{\text{switch2}}(\boldsymbol{s}_1, \boldsymbol{s}_0)$ represent the path cost of switching from the previous motion mode to the original state and the path cost generated by the subsequent motion mode, respectively; $h(\boldsymbol{s}_1)$ is heuristic cost function; p_0 and p_1 represent the motion modes of father node and child node, respectively.

Motion primitives of the in-place rotation mode expand multiple nodes in one expansion and do not repeat the expansion in the same position. So the cost function of holding motion modes is only for the node expansion of the four-wheel Ackermann steering mode and diagonal-move, i.e.,

$$g_{\text{hold}}(\boldsymbol{s}_1, \boldsymbol{s}_0) = \begin{cases} \alpha_1 l + \alpha_2 |\upsilon_1 - \upsilon_0| + & \\ \quad \alpha_3 |\theta_1 - \theta_0|, & p_1 = 1 \\ \beta_1 l + \beta_2 |\upsilon_1 - \upsilon_0| + & \\ \quad \beta_3 |\delta_1 - \delta_0|, & p_1 = 2 \end{cases} \tag{15a}$$

$$g_{\text{switch1}}(\boldsymbol{s}_1, \boldsymbol{s}_0) = \begin{cases} \alpha_5 |\delta_0|, & p_0 = 1 \\ \beta_5 |\delta_0|, & p_0 = 2 \\ \gamma_2, & p_0 = 3 \end{cases} \tag{15b}$$

$$g_{\text{switch2}}(\boldsymbol{s}_1, \boldsymbol{s}_0) = \begin{cases} \alpha_1 l + \alpha_3 |\theta_1 - \theta_0| + \alpha_4, & p_1 = 1 \\ \beta_1 l + \beta_3 |\delta_1 - \delta_0| + \beta_4, & p_1 = 2 \\ \gamma_1 |\theta_1 - \theta_0| + \gamma_2, & p_1 = 3 \end{cases} \tag{15c}$$

where l is determined by the parameters of the motion primitive; α_1, α_2, α_3, α_4, α_5 respectively represent the length weight, reverse weight, turning weight, mode switching weight, and wheel recovery weight when expanding the four-wheel Ackermann steering node; β_1, β_2, β_3, β_4, β_5 are the length weight, reverse weight, orientation change weight, mode switching weight, and wheel recovery weight when expanding diagonal-move nodes; γ_1 and γ_2 are the rotation angle weight and mode switching weight in the in-place rotation mode. Reducing the corresponding weights prioritizes the child nodes associated with the respective motion mode during graph search. For instance, decreasing $\alpha_i, i \in \mathbf{I}_1^5$, increases the preference for selecting nodes under the Ackermann steering mode. Furthermore, if only α_3 is reduced, the child nodes with smaller heading angle variations are preferentially selected within the Ackermann steering mode.

The heuristic cost function plays a crucial role in determining the efficiency and quality of the search process in graph search algorithms. In order to find the shortest path, the proposed algorithm employs a holonomic constraint heuristic cost considering obstacles as the node heuristic cost $h(\boldsymbol{s}_1)$. It is calculated by using the Dijkstra algorithm [26], which is a well-known algorithm for finding the shortest path from the child node $\boldsymbol{s}_1$ to the goal node $\boldsymbol{s}_{\text{goal}}$ in the two-dimensional space.

4.2.2 Node Selection Method

Generally speaking, heuristic graph search algorithms use greedy strategies to select the minimum cost node as the current node $\boldsymbol{s}_{\text{curr}}$. However, such a greedy strategy can often lead to local optimality issues. In our algorithm, this local optimality is likely to lead to unsmooth paths and unreasonable motion modes sequences. Subsequent path optimization algorithms can solve the problem of unsmooth paths, but generating an optimal sequence of motion modes remains a challenge.

In order to alleviate the impact of local optima, traditional methods execute by reducing the weight of the heuristic function. However, this reduction in the heuristic function can lead to a large number of repeated explorations, thereby affecting the search efficiency of the algorithm. To mitigate the impact of local optima while maintaining real-time performance, the proposed algorithm introduces a priority-based optimal node selection method in function `OptimalNodeSelection()` of Algorithm 1. This method enhances the rationality of motion mode selection in the global state by appropriately manipulating the priority of nodes. It accomplishes this without compromising the real-time performance of the search.

The algorithm assigns a counter $\boldsymbol{t}_{\text{p}}$ to each motion mode during node selection. After selecting the node with the minimum cost value, the counter for that node's motion mode is reset to zero, while the counters for other motion modes are incremented by one. If a particular motion mode is not selected during the node extended period, and its counter $\boldsymbol{t}_{\text{p}}$ surpasses a predefined threshold η, in the new iteration of node selection, nodes belonging to that motion mode are given the highest priority. The setting of the threshold η determines the algorithm's ability to handle local optimality. If η is set too small, the algorithm will overly focus on the rationality of the motion mode, potentially impacting the real-time performance. Conversely, if η is too large, the algorithm cannot effectively alleviate the problem of local optima. Based on multiple tests, it is found that setting η within the range of $5 \sim 10$ is reasonable and strikes a balance between addressing local optima and maintaining the real-time performance.

4.3 Goal Node Connection Strategy

Due to the discrete characteristics of graph search algorithms, connecting directly to the goal node solely through node expansion can be challenging. Usually, when searching in the vicinity of the goal node, it is often necessary to generate a path that directly connects to the goal node. One classic method [28] is to use R-S curves [17] for fitting. However, R-S curve fitting is prone to producing paths with large curvature and reverse behavior, which can negatively impact ride comfort.

To enhance the quality of the goal node fitting path, a multi-orientation goal node connection strategy is designed, as shown in Fig. 5. This strategy considers the flexible characteristics of 4WIS vehicles. Firstly, a series of transition nodes with different heading angles are generated at the goal node position $(x_{\text{goal}}, y_{\text{goal}})$. By introducing the in-place rotation mode, the vehicle can smoothly transition between the goal node and the transition node. Then, R-S curve fitting is applied to each goal node and each transition node from the selected node to obtain a series of candidate paths. Finally, the final fitting path is selected through the evaluation function. The lines in Fig. 5 represent all the generated candidate paths, with the black lines representing the selected final fitting path.

When the number of transition nodes generated is n_4, the positions and orientation $(x_{\text{tran}}, y_{\text{tran}}, \theta_{\text{tran}})$ of each transition node are

$$\theta_{\text{tran}} = \theta_{\text{goal}} + i \cdot \frac{2\pi}{n_4 + 1} \quad , i \in \mathbf{I}_1^{n_4} \tag{16a}$$

$$x_{\text{tran}} = x_{\text{goal}} \tag{16b}$$

$$y_{\text{tran}} = y_{\text{goal}} \tag{16c}$$

The proposed method calculates the evaluation value of each fitting path through the evaluation function f_{eva} to select the optimal path. The evaluation function is designed as

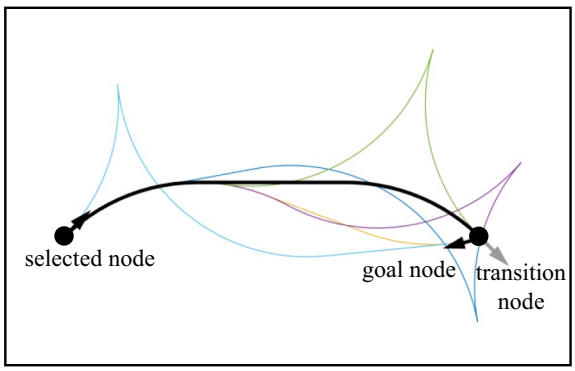


Fig. 5 Goal node connection strategy

$$f_{\text{eva}} = f_{\text{length}} + f_{\text{back}} + f_{\text{rot}} \tag{17}$$

where f_{length} is a penalty for path length; f_{back} is a penalty for reversing behavior on the path, calculated as follows:

$$f_{\text{back}} = \omega_l l_{\text{back}} + \omega_n n_{\text{back}} \tag{18}$$

where ω_l and ω_n are the penalty weights for the length of the reversing path l_{back} and the number of reversing n_{back}, respectively. The penalty for in-place rotation behavior f_{rot} is defined as follows:

$$f_{\text{rot}} = \begin{cases} 0 & \text{, fit to goal node} \\ \omega_\theta \cdot \left|\theta_{\text{tran}} - \theta_{\text{goal}}\right| + \omega_r & \text{, fit to transition node} \end{cases} \tag{19}$$

where ω_θ is the penalty weight for the rotation angle; ω_r is the penalty weight for the in-place rotation behavior.

Due to the penalty term included in the evaluation function, the final selected fitting path demonstrates a shorter length and fewer reverse behaviors compared to other candidate paths. Additionally, our tests show that paths chosen using this method typically exhibit better smoothness. Consequently, this strategy generates superior fitting paths.

Remark 3 The aforementioned goal node connection strategy improves search efficiency and path smoothness when the area near target point is relatively open. However, this strategy does not function as intended in scenarios where the target point is in a narrow space, making in-place rotation infeasible. Notably, "does not function" does not imply that an infeasible path will finally be generated. The collision detection mechanism eliminates impractical goal node connection paths. Instead, the proposed graph search algorithm tends to generate paths that involve in-place rotations at the starting node to avoid collisions.

Remark 4 The primary computational burden of the proposed algorithm lies in the multi-mode collaborative planning phase. Compared to classical graph search algorithms, such as Hybrid A* [28], the proposed multi-mode collaborative planning method incorporates two additional motion modes. Consequently, each iteration of the algorithm expands a greater variety and number of candidate child nodes, which inevitably increases the search time to some extent. However, the proposed method effectively leverages the flexibility of 4WIS vehicles, enabling the exploration of multi-mode child nodes that significantly enhance path generation efficiency in complex scenarios. In contrast, classical graph search methods, constrained by their inability to consider multiple motion modes, often result in unnecessary and frequent reversing maneuvers, making it more challenging to reach the target node. In summary, while the proposed method slightly increases search complexity in open environments, it substantially reduces search time in complex scenarios. This advantage will be demonstrated in the simulations presented in Section 6.

5 Path Smoothing and Trajectory Generation

The graph search algorithm searches for paths through motion primitives with a fixed wheel rotation angle, without considering the curvature discontinuity problem when connecting motion primitives with different wheel rotation angles. This limitation significantly impacts ride comfort. Moreover, during the path search process, various motion primitives are employed, and the step sizes of each motion primitive may vary. This will result in unfixed spacing between each path node, making it challenging for vehicles to follow the path accurately.

In order to achieve a smooth, safe and feasible final trajectory, it is necessary to perform additional processing on the path obtained from the graph search. This processing is divided into three steps: path pre-processing, path optimization, and trajectory generation. In the path preprocessing step, the initial path is segmented, and path nodes are interpolated to ensure smoother transitions. Then, in the path optimization step, the pre-processed path is smoothed and optimized using numerical optimization methods to obtain the final optimized path. Finally, in the trajectory generation step, the speed profile for each path segment is planned, and the speed information is added to the path nodes to obtain the final trajectory.

5.1 Path Pre-processing

The initial path contains path nodes corresponding to multiple motion modes, making it unsuitable for direct path optimization and trajectory generation. To solve this problem, it is necessary to segment the path based on the node expansion characteristics of the three motion modes. The complete path is divided into several segments at the mode switching nodes, categorized as either four-wheel Ackermann paths or diagonal-move paths. The diagonal-move path segment consists of straight lines and does not require numerical optimization. However, for the four-wheel Ackermann steering path, it is necessary to perform path numerical optimization. Each path segment has two endpoints, which can be classified as either in-place rotation endpoints or non-in-place rotation endpoints. The in-place rotation endpoints are particularly useful for relaxing the constraints on endpoint angles, thereby enhancing the effectiveness of the optimization process.

Furthermore, each segmented path requires path interpolation processing. The cubic spline function interpolation method is employed to interpolate the path nodes at equal intervals, facilitating subsequent optimization and speed planning processes.

5.2 Numerical Optimization for Path

After segmenting and interpolating the paths obtained by the proposed graph search algorithm, numerical optimization methods are employed to optimize each four-wheel Ackermann steering path segment.

Path node interpolation addresses the issue of node discontinuity. However, since the initial path is constructed by connecting motion primitives with fixed wheel rotation angles, it can still suffer from problems such as curvature discontinuity and irregular waveform paths. Therefore, it is crucial to optimize path for path tracking and comfortable driving.

5.2.1 Objective Function

Represent the coordinates of each path node $\mathbf{s}_i$ on a path segment as $\{\boldsymbol{Q}_i\} = \{[x_i, y_i]^\top, i \in \mathbf{I}_1^N\}$, where N is the number of path nodes in that path segment. With full consideration of vehicle Kinematics and ride comfort, the objective function is designed as follows:

$$f_{\text{opt}} = \omega_{\text{c}} f_{\text{com}} + \omega_{\text{e}} f_{\text{eff}} + \omega_{\text{d}} f_{\text{dev}} \tag{20}$$

where ω_{c}, ω_{e} and ω_{d} are the weight coefficients corresponding to the comfort term f_{com}, efficiency term f_{eff} and deviation term f_{dev}, respectively.

To improve the comfort of the path, an objective function is designed to improve the smoothness of the path. Calculate the curvature of each path node using its preceding and succeeding nodes, and use the sum of curvature at each path point as the comfort objective function:

$$f_{\text{com}} = \sum_{i=2}^{N-1} \left\| (\boldsymbol{Q}_{i+1} - \boldsymbol{Q}_i) - (\boldsymbol{Q}_i - \boldsymbol{Q}_{i-1}) \right\|_2^2 \tag{21}$$

The entire initial path is obtained through searching, which may result in significant path fluctuations and longer paths. Considering the substantial impact of long paths on traffic efficiency, it is crucial to shorten the path length during the optimization process. To achieve this, an objective function is designed based on the sum of distances between each path node. With the utilization of this objective function, not only can the path be shortened, but the distances between path nodes can also be further adjusted, i.e.,

$$f_{\text{eff}} = \sum_{i=1}^{N-1} \left\| (\boldsymbol{Q}_{i+1} - \boldsymbol{Q}_i) \right\|_2^2 \tag{22}$$

While ensuring comfort and efficiency, the optimized path should also be kept as close as possible to the initial path. Design path deviation objective function as follows:

$$f_{\text{dev}} = \sum_{i=1}^{N} \left\| (\boldsymbol{Q}_i - \boldsymbol{Q}_i^{\text{ref}}) \right\|_2^2 \tag{23}$$

where $\boldsymbol{Q}_i^{\text{ref}}$ represents the unoptimized initial node corresponding to node $\boldsymbol{Q}_i$.

Adjusting the three weights enables the planned path to exhibit different driving styles and adapt to varying environments. Specifically, increasing ω_{c} enhances the smoothness of the path, while increasing ω_{e} prioritizes shorter path lengths. The weight ω_{d} reflects the level of confidence in the initial graph search results. A larger ω_{d} yields an optimized path that more closely aligns with the coarse search outcome. For instance, when applying a 4WIS chassis to passenger vehicles, comfort is a key consideration, and a higher ω_{c} is recommended. Conversely, if the 4WIS chassis is deployed in autonomous transport scenarios, such as ports or open-pit mines, increasing ω_{e} is advantageous to minimize transport distances and times, thereby achieving greater economic efficiency.

5.2.2 Constraint Conditions

The optimized path should not only minimize the objective function, but also satisfy endpoint constraints, feasible region constraints and curvature constraints. To avoid affecting other path segments, it is necessary to constrain the positions of the start and terminal points of the optimized path, i.e.,

$$\boldsymbol{Q}_1 = \boldsymbol{Q}_1^{\text{ref}} \tag{24a}$$

$$\boldsymbol{Q}_N = \boldsymbol{Q}_N^{\text{ref}} \tag{24b}$$

For the convenience of calculation, the constraint on the endpoint orientation is achieved by fixing the position of its adjacent points. In addition, when an endpoint is the in-place rotation endpoint, unconstrained endpoint orientation is beneficial for obtaining a better path, that is

$$\boldsymbol{Q}_2 = \boldsymbol{Q}_1^{\text{ref}} + \Delta l \cdot \begin{bmatrix} \sin\theta_1 \\ \cos\theta_1 \end{bmatrix} \quad , \text{ if } p_1 \neq 3 \tag{25a}$$

$$\boldsymbol{Q}_{N-1} = \boldsymbol{Q}_N^{\text{ref}} - \Delta l \cdot \begin{bmatrix} \sin\theta_N \\ \cos\theta_N \end{bmatrix} \quad , \text{ if } p_N \neq 3 \tag{25b}$$

where θ_1 and θ_N are the angles of the start and terminal points on the initial path, respectively; p_1 and p_N respectively

represent the motion modes of the start and terminal points. When $p = 3$ is used, it indicates that it is the in-place rotation endpoint.

The optimized path should maintain the maneuver determined by the initial path. Construct feasible regions for each path node, and generate corresponding optimized path nodes within the feasible region, i.e.,

$$\{\boldsymbol{L}_i\} \leq \{\boldsymbol{Q}_i - \boldsymbol{Q}_i^{\text{ref}}\} \leq \{\boldsymbol{U}_i\} \tag{26}$$

where $\boldsymbol{L}_i = \{[-r_i, -r_i]^\top\}$ and $\boldsymbol{U}_i = \{[r_i, r_i]^\top\}$ represent the lower and upper bounds of the optimization range of each path node, which are determined by the parameter r_i. In addition, the construction of the feasible region can also be used to ensure the security of the optimized path. The initial value r_i^{init} of the feasible region corresponding to each node is set to be the same, and it is reduced as the optimization iteration progresses to ensure collision avoidance. Describe in detail in the subsequent Section 5.2.4.

When optimizing the path generated by the four-wheel Ackermann steering mode, the path needs to satisfy the vehicle Kinematics and constrain the curvature of each path node, that is

$$\left\|(\boldsymbol{Q}_{i+1} - \boldsymbol{Q}_i) - (\boldsymbol{Q}_i - \boldsymbol{Q}_{i-1})\right\|_2^2 \leq \left((\Delta l)^2 k_{\max}\right)^2 \tag{27}$$

where Δl represents the distance between path nodes after interpolation; $k_{\max}$ is the maximum curvature of the constraint.

5.2.3 Optimization Problem

Through the objective function and constraint conditions constructed above, the path numerical optimization problem can be represented as

$$\begin{aligned}
\min \quad & f_{\text{opt}} = \omega_{\text{c}} f_{\text{com}} + \omega_{\text{e}} f_{\text{eff}} + \omega_{\text{d}} f_{\text{dev}} \\
\text{s.t.} \quad & \boldsymbol{Q}_1 = \boldsymbol{Q}_1^{\text{ref}} \\
& \boldsymbol{Q}_N = \boldsymbol{Q}_N^{\text{ref}} \\
& \boldsymbol{Q}_2 = \boldsymbol{Q}_1^{\text{ref}} + \Delta l \cdot [\sin\theta_1, \cos\theta_1]^\top, \text{if } p_1 \neq 3 \\
& \boldsymbol{Q}_{N-1} = \boldsymbol{Q}_N^{\text{ref}} - \Delta l \cdot [\sin\theta_N, \cos\theta_N]^\top, \text{if } p_N \neq 3 \\
& \{\boldsymbol{L}_i\} \leq \{\boldsymbol{Q}_i - \boldsymbol{Q}_i^{\text{ref}}\} \leq \{\boldsymbol{U}_i\} \\
& \left\|(\boldsymbol{Q}_{i+1} - \boldsymbol{Q}_i) - (\boldsymbol{Q}_i - \boldsymbol{Q}_{i-1})\right\|_2^2 \leq \left((\Delta l)^2 k_{\max}\right)^2
\end{aligned} \tag{28}$$

To expedite the solution process, we linearize the nonlinear curvature constraint, as solving nonlinear optimization problems can be time-consuming. Afterwards, the original problem is transformed into a QP problem. Compared to directly solving the nonlinear optimization problem, we can better satisfy the real-time planning requirements by converting it into a QP problem.

5.2.4 Collision Avoidance

In the optimization problem, the consideration of obstacles was not included. Instead, feasible regions were adopted to avoid collisions resulting from excessive deviation of path nodes. However, since the initial feasible region is set relatively large and obstacles are not taken into account, the optimized path is likely to collide. To ensure the safe operation of vehicles, particularly in narrow scenarios like parallel parking, precise path collision avoidance is necessary.

To solve this problem, the iterative loop approach [22] is adopted. The entire optimization and collision avoidance flowchart is shown in Fig. 6. After solving each optimization problem, we perform a collision check on the path. If a collision occurs at the i th path node, we proportionally reduce the corresponding feasible regions $\boldsymbol{L}_i$ and $\boldsymbol{U}_i$ by a factor of $\xi < 1$. Subsequently, we re-solve the optimization problem while keeping the objective function and other constraints unchanged. Through multiple iterations, as the feasible region decreases, the collision point gradually moves closer to its initial position. As the initial search path is known to be safe and collision-free, we can ultimately obtain a smooth and collision-free path through the iterative process.

5.3 Trajectory Generation

After generating and optimizing the path, a complete trajectory planning task should also include speed planning for the path. In this section, we need to generate a speed profile for the above path. Considering that the method proposed in this paper primarily focuses on parking and other complex unstructured scenes, its speed should be limited to a lower range. Given that the path may involve various behaviors, such as changing the vehicle's direction and switching between different motion modes, we assume that the path always stops at the switching position to enhance ride comfort. Then, speed planning is conducted on each individual path segment using a trapezoidal speed planning approach.

Considering that the vehicle operates at low speeds in narrow environments where speed does not vary significantly, we assume that the acceleration a is constant. The maximum vehicle speed is $v_{\max}$ and S is the length of the path segment. If S is less than $\frac{v_{\max}^2}{a}$, the trajectory contains a uniform acceleration segment and a uniform deceleration segment. The speed $v(\boldsymbol{s}_i)$ of node $\boldsymbol{s}_i$ is as follows:

$$v(\boldsymbol{s}_i) = \begin{cases} \sqrt{aS_i} & , S_i \in (0, S/2) \\ \sqrt{a(S - S_i)} & , S_i \in (S/2, S) \end{cases} \tag{29}$$

where S_i represents the distance of node $\boldsymbol{s}_i$ from the starting point on the path.

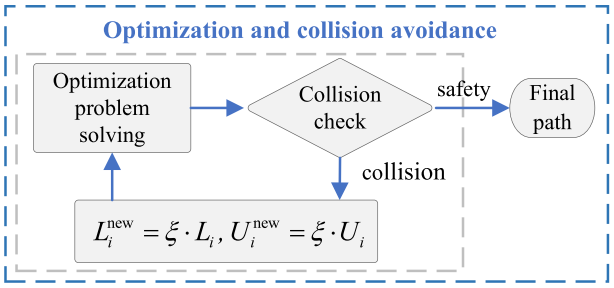


Fig. 6 Optimization and collision avoidance flow chart

If S is greater than $\frac{v_{max}^2}{a}$, then the trajectory includes a uniform acceleration segment, a uniform velocity segment, and a uniform deceleration segment. The speed $v(\boldsymbol{s}_i)$ of node $\boldsymbol{s}_i$ is calculated as follows:

$$v(\boldsymbol{s}_i) = \begin{cases} \sqrt{aS_i} & , S_i \in \left(0, \frac{v_{max}^2}{2a}\right) \\ v_{max} & , S_i \in \left(\frac{v_{max}^2}{2a}, S - \frac{v_{max}^2}{2a}\right) \\ \sqrt{a(S - S_i)} & , S_i \in \left(S - \frac{v_{max}^2}{2a}, S\right) \end{cases} \tag{30}$$

6 Simulation

This section presents simulations to demonstrate the superiority and effectiveness of the proposed method. Section 6.1 and 6.2 respectively present the settings and results of the simulation.

6.1 Simulation Settings

Simulations are conducted in MATLAB 2022b. Two specific scenarios are designed for the simulations: the first one involves a narrow passage with complex obstacles, and the second one focuses on a narrow parallel parking scene. These scenarios are intended to evaluate the effectiveness of the proposed method in challenging and complex environments.

To validate the superiority of the proposed method, two other methods are employed for comparison. The first comparison method utilizes a Hybrid-A* algorithm [28] to plan the initial path, followed by optimization using the QP optimization algorithm. The second type is the CC-RRT* algorithm [37], which integrates R-S curves with the RRT* algorithm. This integration allows the CC-RRT* algorithm to generate curvature continuous paths. The simulation parameters of this method are shown in Table 1. To ensure fairness in the comparison, the parameters used in the comparison algorithms are set to be essentially the same as those used in this paper's algorithm.

6.2 Simulation Result

6.2.1 Narrow Passage with Complex Obstacles

Case 1, referred to as the narrow passage with complex obstacles, aims to demonstrate the ability of the proposed algorithm to leverage the 4WIS vehicle's high flexibility in such constrained spaces. Fig. 7 shows two representative planning cases of the algorithm in a narrow environment, characterized by numerous obstacles and tight bends. The curvature comparison of its planned trajectory is depicted in Fig. 8. As shown in the figure, the proposed method can flexibly utilize various motion modes to navigate through narrow spaces, effectively generating smooth and comfortable motion trajectories. However, the Hybrid-A* algorithm and CC-RRT* algorithm require maneuvers such as reverse and high curvature turns to navigate out of restricted areas, thereby reducing ride comfort.

Table 1 Simulation parameters table

Parameters	Description	Setting	Parameters	Description	Setting
L	Wheelbase of vehicle	2.6 m	L_f, L_r	Front and rear hang length of vehicle	0.7 m
D	Width of vehicle	2.0 m	$\delta_{min}, \delta_{max}$	Minimum and maximum wheel rotation angles	-45°, 45°
l	Motion primitive step size	[0.3 m, 0.9 m]	n_1, n_2	Number of wheel rotation angles	5,4
n_3	Number of angle values	8	$\alpha_1, \alpha_2, \alpha_3, \alpha_4$	Weight in cost function (15a)	1.0, 1.0, 0.2, 0, 0.2
$\beta_1, \beta_2, \beta_3, \beta_4$	Weight in cost function (15a) and (15b)	1.0, 1.0, 0.2, 0.5, 0.2	γ_1, γ_2	Weight in cost function (15b)	0.2, 1.0
n_4	Number of transition node	8	$\omega_l, \omega_n, \omega_\theta, \omega_r$	Weight in evaluation function (18) and (19)	0.5, 1.0, 0.2, 2.0
$\omega_c, \omega_e, \omega_d$	Weight in objective function (20)	100, 1, 1	Δl	Path nodes interpolation step size	0.15
r_i^{init}	Initial size of feasible region	1 m	ξ	Feasible region scaling ratio	0.5

Table 2 Simulation results table

Case	Method	Planning time (s)	Path length (m)	Reverse length (m)	Number of path
Case 1	proposed overall method	20.07	64.30	0	3
	Hybrid-A*+QP	56.75	69.13	7.20	5
	CC-RRT*	1718.62	100.80	42.17	7
Case 2	proposed overall method	8.74	47.29	11.69	3
	Hybrid-A*+QP	17.86	56.12	28.16	6
	CC-RRT*	532.88	65.03	37.04	4
Case 3	proposed collaborative planning method	1.68	27.43	0	1
	Hybrid-A*	1.31	27.43	0	1

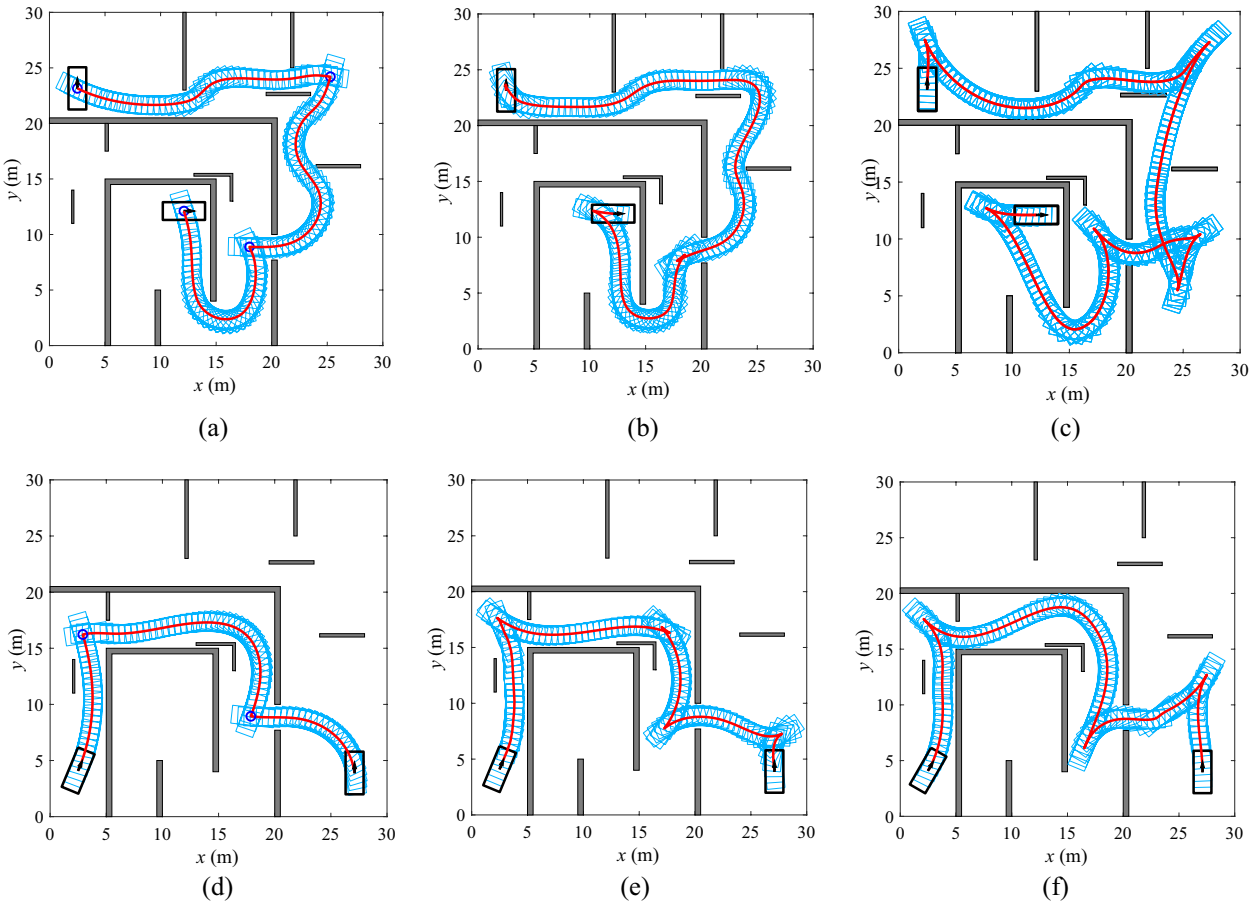


Fig. 7 Visualization of simulation results in narrow passage with complex obstacles. The red path line represents the four-wheel Ackermann steering mode path, and the blue circle represents the in-place rotation node. **a** Results of proposed method in Case 1; **b** Results of hybrid-A*+QP method in Case 1; **c** Results of CC-RRT* algorithm in Case 1; **d** Results of proposed method in Case 2; **e** Results of hybrid-A*+QP method in Case 2; **f** Results of CC-RRT* algorithm in Case 2

Other comparison results are shown in Table 2. The proposed method exhibits several key advantages, including faster planning time, avoidance of reverse behavior, and a small number of path segments. This is because, when a narrow bend is encountered, the proposed method explores from the in-place rotation nodes through the designed optimal node selection strategy. After adjusting the vehicle orientation, the trajectory for navigating the narrow curve can be quickly determined. It can be noted that the planning time of the CC-RRT* algorithm is much larger than that of other algorithms. This is because random sampling is difficult to deal with narrow areas, resulting in a lot of

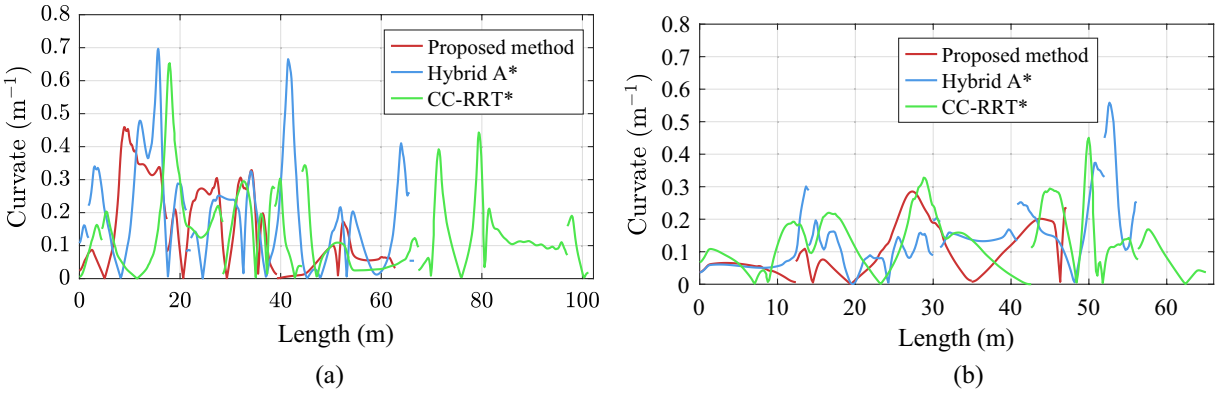


Fig. 8 Comparison of path curvature generated in narrow passage with complex obstacles. **a** Curvature comparison of Case 1; **b** Curvature comparison of Case 2

useless explorations of the algorithm. In addition, our algorithm hardly performs the diagonal-move motion mode in the planning results, because it is not the optimal solution in most cases.

6.2.2 Narrow Parallel Parking Scene

Case 2, referred to as parallel parking in narrow spaces, has always been a challenge for vehicle driving. A typical front-wheel steering vehicle may need to undergo several vehicle pose adjustments in a narrow parallel parking space before successfully exiting it. The emergence of 4WIS vehicles can effectively solve this problem. By utilizing the diagonal-move mode, the vehicle can easily navigate out of the narrow parallel parking space.

The comparative simulation in narrow parallel parking sceneis shown in Fig. 9. Since the RRT algorithm is rarely used in parking scenarios, we only use the Hybrid-A*+QP method for comparison. In this situation, the proposed method effectively addresses the challenge of parallel parking. As illustrated in Fig. 9(a), the vehicle is translated out of the parking space through the diagonal-move mode, and then the trajectory connected to the goal node is easily determined through the four-wheel Ackermann steering mode. At this time, the vehicle has only undergone one movement mode switch and one driving direction switch. The simulation results of the Hybrid-A* algorithm, as shown in Fig. 9(b), demonstrate that due to the narrow parking space, the vehicle still encounters difficulties in smoothly exiting the parking space even after several adjusting its pose. Throughout the entire process, the vehicle underwent four directional changes and generated five path segments. Fig. 10 shows the variation of curvature with the length of the trajectory during the entire parking process. The curvature of the diagonal-move mode path is represented by 0. It can be seen that the trajectory curvature obtained by the proposed method is small.

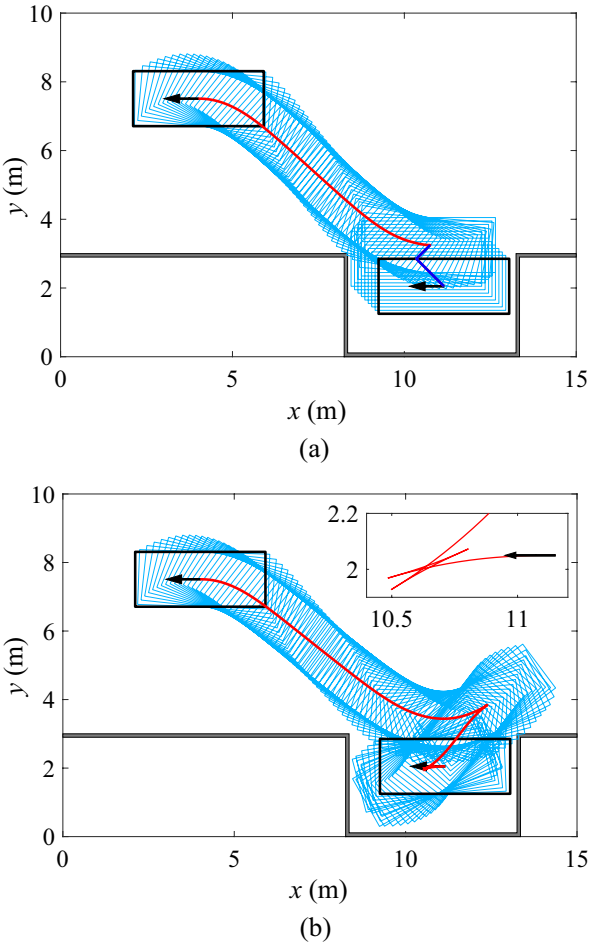


Fig. 9 Visualization of simulation results in narrow parallel parking scene. The dark blue path line represents the diagonal-move path. **a** Results of proposed method; **b** Results of hybrid-A* algorithm

6.2.3 Open Area with Few Obstacles

Case 3 involves an open area scenario where the vehicle can traverse with ease. To evaluate computational complexity,

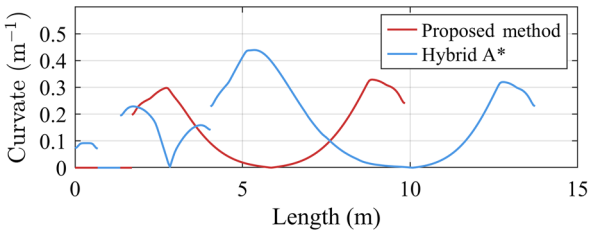


Fig. 10 Comparison of path curvature generated in narrow parallel parking scene

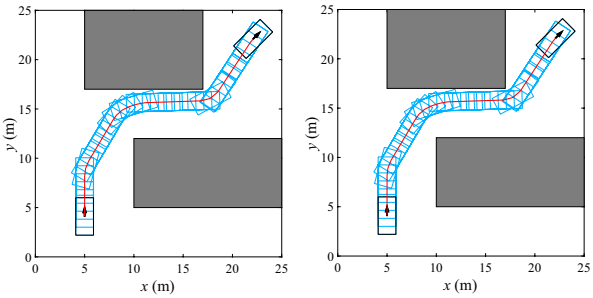


Fig. 11 Visualization of simulation results in open area scene. **a** Results of proposed collaborative method; **b** Results of hybrid-A* algorithm

the proposed multi-mode collaborative planning method is compared against the classical hybrid A* algorithm [28]. The resulting paths are illustrated in Fig. 11, and the associated statistical data is summarized in Table 2. In this straightforward scenario, due to the absence of complex obstacles, the 4WIS vehicle exclusively utilizes the Ackermann steering mode throughout the traversal, reaching the goal without requiring any reversing maneuvers. The results indicate that the proposed method generates a path highly similar to that of the hybrid A* algorithm, albeit with a slightly increased computation time, which aligns with the observations in Remark 4. Nevertheless, the increase in computation time remains within an acceptable range, underscoring the practical feasibility of the proposed approach.

7 Conclusion

This study has proposed a trajectory planning method incorporating multiple motion mode sequence decision-making for the 4WIS vehicle. By introducing the multi-mode node expansion strategy, optimal node selection strategy, and goal node connection strategy, the multi-mode collaborative planning algorithm has been designed, which realized the ingenious integration of path planning and mode decision-making. Specifically, we have designed multiple motion primitives for path search and developed an optimal node selection strategy that allows vehicles to flexibly switch motion modes in different scenarios. Additionally, the searched path has been segmented at the mode switching node, and numerical optimization has been performed on the four-wheel Ackermann steering mode path segment to enhance path quality. Finally, the incorporation of speed profiles has enabled the generation of a final trajectory for each path segment. The simulation results show that the proposed method effectively guides 4WIS vehicles in complex environments and makes rational use of multiple motion modes during the planning process. Compared with other algorithms, our method exhibits higher planning efficiency and yields superior path quality.

This study focuses on trajectory planning tasks in environments with static obstacles. However, it has certain limitations that warrant further exploration in future research:

(1) The current study does not address dynamic obstacle avoidance. In future work, we will explore trajectory planning for 4WIS vehicles in the presence of dynamic obstacles.
(2) The present research assumes that the vehicle decelerates to a complete stop during motion mode transitions. In practice, smooth mode transitions without stopping could be studied in greater detail.
(3) The kinematic modeling of 4WIS vehicles could be refined to incorporate additional motion modes, thereby maximizing their flexibility and advantages.
(4) The computational real-time performance needs further enhancement to enable large-scale deployment and application in 4WIS vehicles.

These aspects will guide our efforts toward extending the applicability and robustness of the proposed approach in future studies.

Acknowledgements This work was supported by National Natural Science Foundation of China (52222216, 52202493, 52275105), Hunan Provincial NSF of China (2023JJ10008), Young Elite Scientists Sponsorship Program by CAST (2022QNRC001).

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Hang, P., Chen, X., Fang, S., et al.: Robust control for four-wheel-independent-steering electric vehicle with steer-by-wire system. Int. J. Automot. Technol. **18**(5), 785–797 (2017)

2. Ye, Y., He, L., Zhang, Q.: Steering control strategies for a four-wheel-independent-steering bin managing robot. IFAC-PapersOnLine **49**(16), 39–44 (2016)
3. Hang, P., Chen, X.: Towards autonomous driving,: Review and perspectives on configuration and control of four-wheel independent drive/steering electric vehicles. Actuators **10**, 184 (2021)
4. Liu, T., Shen, Y., Wang, K.: Path tracking control for autonomous truck with dual modular chassis. Automot. Innov. **6**(4), 558–570 (2023)
5. Lai, X., Chen, X.B., Wu, X.J., et al.: A study on control system for four-wheels independent driving and steering electric vehicle. Appl. Mech. Mater. **701**, 807–811 (2015)
6. Chen, X., Luo, F., Hang, P., et al.: Steering mode switch control of four-wheel-independent-steering electric vehicle. In: Society of Automotive Engineers (SAE)-China Congress, Springer, pp 437–453 (2017)
7. Parhi, D.R., Mohanty, P.K.: Iwo-based adaptive neuro-fuzzy controller for mobile robot navigation in cluttered environments. Int. J. Adv. Manuf. Technol. **83**(9), 1607–1625 (2016)
8. Kim, M., Ahn, J., Park, J.: Targettree-RRT*: continuous-curvature path planning algorithm for autonomous parking in complex environments. IEEE Trans. Autom. Sci. Eng. **21**(1), 606–617 (2022)
9. González, D., Pérez, J., Milanés, V., et al.: A review of motion planning techniques for automated vehicles. IEEE Trans. Intell. Transp. Syst. **17**(4), 1135–1145 (2015)
10. Paden, B., Čáp, M., Yong, S.Z., et al.: A survey of motion planning and control techniques for self-driving urban vehicles. IEEE Trans. Intell. Veh. **1**(1), 33–55 (2016). https://doi.org/10.1109/TIV.2016.2578706
11. LaValle, S.M., Kuffner, J.J., Jr.: Randomized kinodynamic planning. Int. J. Robot. Res. **20**(5), 378–400 (2001)
12. Kavraki, L.E., Svestka, P., Latombe, J.C., et al.: Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE Trans. Robot. Autom. **12**(4), 566–580 (2002)
13. Kuffner, J.J., LaValle, S.M.: Rrt-connect: An efficient approach to single-query path planning. In: Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065), IEEE, pp 995–1001 (2000)
14. Karaman, S., Frazzoli, E.: Sampling-based algorithms for optimal motion planning. Int. J. Robot. Res. **30**(7), 846–894 (2011)
15. Wang, J., Chi, W., Li, C., et al.: Efficient robot motion planning using bidirectional-unidirectional rrt extend function. IEEE Trans. Autom. Sci. Eng. **19**(3), 1859–1868 (2021)
16. Dubins, L.E.: On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents. Am. J. Math. **79**(3), 497–516 (1957)
17. Reeds, J., Shepp, L.: Optimal paths for a car that goes both forwards and backwards. Pac. J. Math. **145**(2), 367–393 (1990)
18. Tehrani, H., Do, Q.H., Egawa, M., et al.: General behavior and motion model for automated lane change. In: 2015 IEEE Intelligent Vehicles Symposium (IV), IEEE, pp 1154–1159, (2015)
19. Usenko, V., Von Stumberg, L., Pangercic, A., et al.: Real-time trajectory replanning for mavs using uniform b-splines and a 3d circular buffer. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, pp 215–222, (2017)
20. Ding, W., Zhang, L., Chen, J., et al.: Safe trajectory generation for complex urban environments using spatio-temporal semantic corridor. IEEE Robot. Autom. Lett. **4**(3), 2997–3004 (2019)
21. Wang, Y., Cao, X., Hu, Y.: A trajectory planning method of automatic lane change based on dynamic safety domain. Autom. Innov. **6**(3), 466–480 (2023)
22. Zhou, J., He, R., Wang, Y., et al.: Autonomous driving trajectory optimization with dual-loop iterative anchoring path smoothing and piecewise-jerk speed optimization. IEEE Robot. Autom. Lett. **6**(2), 439–446 (2020)
23. Schulman, J., Duan, Y., Ho, J., et al.: Motion planning with sequential convex optimization and convex collision checking. Int. J. Robot. Res. **33**(9), 1251–1270 (2014)
24. Zhang, X., Liniger, A., Borrelli, F.: Optimization-based collision avoidance. IEEE Trans. Control Syst. Technol. **29**(3), 972–983 (2020)
25. Zhang, T., Fu, M., Song, W., et al.: Trajectory planning based on spatio-temporal map with collision avoidance guaranteed by safety strip. IEEE Trans. Intell. Transp. Syst. **23**(2), 1030–1043 (2020)
26. Dijkstra, E.W.: A note on two problems in connexion with graphs. In: Edsger Wybe Dijkstra: his life, work, and legacy. p 287–290, (2022)
27. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. Syst. Sci. Cybern. **4**(2), 100–107 (1968). https://doi.org/10.1109/TSSC.1968.300136
28. Dolgov, D., Thrun, S., Montemerlo, M., et al.: Path planning for autonomous vehicles in unknown semi-structured environments. Int. J. Robot. Res. **29**(5), 485–501 (2010)
29. Montemerlo, M., Becker, J., Bhat, S., et al.: Junior: the stanford entry in the urban challenge. J. Field Robot. **25**(9), 569–597 (2008)
30. Ziegler, J., Werling, M., Schroder, J.: Navigating car-like robots in unstructured environments using an obstacle sensitive cost function. In: 2008 IEEE intelligent vehicles symposium, IEEE, pp 787–791, (2008)
31. Likhachev, M., Gordon, G.J., Thrun, S.: ARA*: Anytime A* with provable bounds on sub-optimality. Advances in neural information processing systems 16, (2003)
32. Ferguson, D., Stentz, A.: Using interpolation to improve path planning: the field D* algorithm. J. Field Robot. **23**(2), 79–101 (2006)
33. Zhang, Y., Chen, G., Hu, H., et al.: Hierarchical parking path planning based on optimal parking positions. Autom. Innov. **6**(2), 220–230 (2023)
34. Zhang, X., Huang, Y., Wang, S., et al.: Motion planning and tracking control of a four-wheel independently driven steered mobile robot with multiple maneuvering modes. Front. Mech. Eng. **16**(3), 504–527 (2021)
35. Xu, F., Liu, X., Chen, W., et al.: Dynamic switch control of steering modes for four wheel independent steering rescue vehicle. IEEE Access **7**, 135595–135605 (2019)
36. Hang, P., Huang, C., Hu, Z., et al.: Decision making of connected automated vehicles at an unsignalized roundabout considering personalized driving behaviours. IEEE Trans. Veh. Technol. **70**(5), 4051–4064 (2021)
37. Banzhaf, H., Palmieri, L., Nienhüser, D., et al.: Hybrid curvature steer: A novel extend function for sampling-based nonholonomic motion planning in tight environments. In: 2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC), IEEE, pp 1–8, (2017)

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.